

A decorative graphic consisting of a thin yellow circle on the left side, partially overlapping a horizontal bar. The bar has a yellow-to-white gradient and is flanked by a large black '[' bracket on the left and a large yellow ']' bracket on the right.

Langage de Programmation C

pointeurs et Tableaux

Département de la formation préparatoire ST
Ecole Nationale Polytechnique d'Oran

Pointeurs et tableaux

- Le nom d'un tableau **T** représente l'adresse de son premier élément (**T=&T[0]**).
- En simplifiant, nous pouvons retenir que *le nom d'un tableau est un **pointeur constant** sur le premier élément du tableau.*
- En déclarant un tableau **T** et un pointeur **P** du même type, l'instruction **P=T** fait pointer **P** sur le premier élément de **T** (**P=&T[0]**) et crée une liaison entre **P** et le tableau **T**.



Pointeurs et tableaux

- l'instruction **P=T** fait pointer **P** sur le premier élément de **T** ($P=\&T[0]$) et crée une liaison entre **P** et le tableau **T**.



A partir de là, on peut manipuler le tableau **T** en utilisant **P**, en effet :

- **P** pointe sur **T[0]** et ***P** désigne **T[0]**
- **P+1** pointe sur **T[1]** et ***(P+1)** désigne **T[1]**
-
- **P+i** pointe sur **T[i]** et ***(P+i)** désigne **T[i]**

Pointeurs et tableaux : exemple

Exemple:

```
short x, A[7]={5,0,9,2,1,3,8};
```

```
short *P;
```

```
P=A;
```

```
x=*(P+5);
```

- Le compilateur obtient l'adresse $P+5$ en ajoutant $5 * \text{sizeof}(\text{short}) = 10$ octets à l'adresse dans P
- D'autre part, les composantes du tableau sont stockées à des emplacements contigus et $\&A[5] = \&A[0] + \text{sizeof}(\text{short}) * 5 = A + 10$
- Ainsi, x est égale à la valeur de $A[5]$ ($x = A[5]$)

[*Attention !*]

- Il existe toujours une différence essentielle entre un pointeur et le nom d'un tableau:
 - Un *pointeur* est une variable,
donc des opérations comme $P = A$ ou $P++$ sont permises.
 - Le *nom d'un tableau* est une constante,
donc des opérations comme $A = P$ ou $A++$ sont impossibles.

saisie et affichage d'un tableau

(avec le formalisme pointeur)

Version 1:

```
main()
{ float T[100] , *pt;
  int i,n;
  do {printf("Entrez n \n " );
    scanf(" %d" ,&n);
    }while(n<0 ||n>100);
  pt=T;
  for(i=0;i<n;i++)
    { printf ("Entrez T[%d] \n ",i );
      scanf(" %f" , pt+i);
    }
  for(i=0;i<n;i++)
    printf (" %f \t",*(pt+i));
}
```

Version 2: sans utiliser i

```
main()
{ float T[100] , *pt;
  int n;
  do {printf("Entrez n \n " );
    scanf(" %d" ,&n);
    }while(n<0 ||n>100);
  for(pt=T;pt<T+n;pt++)
    { printf ("Entrez T[%d] \n ",pt-T );
      scanf(" %f" , pt);
    }
  for(pt=T;pt<T+n;pt++)
    printf (" %f \t",*pt);
}
```

ON RESUME

Sources d'erreurs

Un bon nombre d'erreurs lors de l'utilisation de C provient de la confusion entre soit contenu et adresse, soit pointeur et variable. Revoyons donc les trois types de déclarations que nous connaissons jusqu'ici et résumons les possibilités d'accès aux données qui se présentent.

Les variables et leur utilisation

int A; déclare une **variable simple** du type **int**

A désigne *le contenu de A*

&A désigne l'adresse de A

int B[]; déclare un **tableau** d'éléments du type **int**

- **B** désigne *l'adresse de la première composante de B*. (Cette adresse est toujours constante)
- **B[i]** désigne le contenu de la composante i du tableau
- **&B[i]** désigne l'adresse de la composante i du tableau

en utilisant le formalisme pointeur:

- **B+i** désigne l'adresse de la composante i du tableau
- ***(B+i)** désigne le contenu de la composante i du tableau

[ON RESUME]

`int *P;` déclare un *pointeur* sur des éléments du type `int`.

- **P** peut pointer sur des variables simples du type `int` ou sur les composantes d'un tableau du type `int`.
- **P** désigne *l'adresse contenue dans P* (Cette adresse est variable)
- ***P** désigne le contenu de l'adresse dans P

Si P *pointe* dans un *tableau*, alors

- **P** désigne l'adresse de la première composante
- **P+i** désigne l'adresse de la i-ième composante derrière P
- ***(P+i)** désigne le contenu de la i-ième composante derrière P

exercice

- Soit P un pointeur qui 'pointe' sur un tableau A:
`int A[] = {12, 23, 34, 45, 56, 67, 78, 89, 90};`
`int *P; P = A;`
- Quelles valeurs ou adresses fournissent ces expressions:
 - a) `*P+2` $\Rightarrow$ la **valeur 14**
 - b) `*(P+2)` $\Rightarrow$ la **valeur 34**
 - c) `&P+1` $\Rightarrow$ **l'adresse** du pointeur derrière le pointeur P (rarement utilisée)
 - d) `&A[4]-3` $\Rightarrow$ **l'adresse** de la composante **A[1]**
 - e) `A+3` $\Rightarrow$ **l'adresse** de la composante **A[3]**
 - f) `&A[7]-P` $\Rightarrow$ la **valeur (indice) 7**
 - g) `P+(*P-10)` $\Rightarrow$ **l'adresse** de la composante **A[2]**
 - h) `*(P+*(P+8)-A[7])` $\Rightarrow$ la **valeur 23**

[exercice]

- Ecrire un programme qui lit un entier X et un tableau A du type **int** au clavier et élimine toutes les occurrences de X dans A en tassant les éléments restants. Le programme utilisera les pointeurs P1 et P2 pour parcourir le tableau.

```

#include <stdio.h>
main()
{
    /* Déclarations */
    int A[50];    /* tableau donné */
    int N;        /* dimension du tableau */
    int X;        /* valeur à éliminer */
    int *P1, *P2; /* pointeurs d'aide */

    /* Saisie des données */
    printf("Dimension du tableau (max.50) : ");
    scanf("%d", &N );
    for (P1=A; P1<A+N; P1++)
    {
        printf("Elément %d : ", P1-A);
        scanf("%d", P1);
    }

    printf("Introduire l'élément X à éliminer du tableau : ");
    scanf("%d", &X );

    /* Affichage du tableau */
    for (P1=A; P1<A+N; P1++)
        printf("%d ", *P1);
    printf("\n");

```

```

    /* Effacer toutes les occurrences de X */
    /* et comprimer : */
    /* Copier tous les éléments de P1 vers P2 */
    /* et augmenter P2 pour tous */
    /* les éléments différents de X. */
    for (P1=P2=A; P1<A+N; P1++)
    {
        *P2 = *P1;
        if (*P2 != X)
            P2++;
    }

    /* Nouvelle dimension de A */
    N = P2-A;
    if (N==0)
        printf("le tableau resultat est vide ");
    else
        /* Edition du résultat */
        for (P1=A; P1<A+N; P1++)
            printf("%d ", *P1);
        printf("\n");
    /* return 0;*/
    getch();
}

```

Pointeurs et tableaux à deux dimensions

- *Le nom d'un tableau **A** à deux dimensions est un pointeur **constant** sur le premier élément du tableau càd **A[0][0]**.*
- En déclarant un tableau **A[n][m]** et un pointeur **P** du même type, on peut manipuler le tableau **A** en utilisant le pointeur **P** en faisant pointer P sur le premier élément de **A** (**P=&A[0][0]**), Ainsi :
 - **P** pointe sur **A[0][0]** et ***P** désigne **A[0][0]**
 - **P+1** pointe sur **A[0][1]** et ***(P+1)** désigne **A[0][1]**
 -
 - **P+M** pointe sur **A[1][0]** et ***(P+M)** désigne **A[1][0]**
 -
 - **P+i*M** pointe sur **A[i][0]** et ***(P+i*M)** désigne **A[i][0]**
 -
 - **P+i*M+j** pointe sur **A[i][j]** et ***(P+i*M+j)** désigne **A[i][j]**

Pointeurs : saisie et affichage d'une matrice

```
#define N 10
#define M 20
main( )
{ int i, j, A[N][M], *pt;
  pt=&A[0][0];
  for(i=0;i<N;i++)
    for(j=0;j<M;j++)
      { printf ("Entrez A[%d][%d]\n ",i,j );
        scanf(" %d" , pt+i*M+j);
      }
  for(i=0;i<N;i++)
    { for(j=0;j<M;j++)
      printf (" %d \t",*(pt+i*M+j));
      printf ("\n");
    }
}
```

[Pointeurs et tableaux : remarques]

En C, on peut définir :

- **Un tableau de pointeurs :**

Ex : `int *T[10];` //déclaration d'un tableau de 10 pointeurs d'entiers

- **Un pointeur de tableaux :**

Ex : `int (*pt)[20];` // déclaration d'un pointeur sur des tableaux de 20 éléments

- **Un pointeur de pointeurs :**

Ex : `int **pt;` //déclaration d'un pointeur pt qui pointe sur des pointeurs d'entiers

Allocation dynamique de mémoire

- Quand on déclare une variable dans un programme, on lui réserve implicitement un certain nombre d'octets en mémoire. Ce nombre est connu avant l'exécution du programme
- Or, il arrive souvent qu'on ne connaît pas la taille des données au moment de la programmation. On réserve alors l'espace maximal prévisible, ce qui conduit à un gaspillage de la mémoire
- Il serait souhaitable d'allouer la mémoire en fonction des données à saisir (par exemple la dimension d'un tableau)
- Il faut donc un moyen pour allouer la mémoire lors de l'exécution du programme : c'est l'allocation dynamique de mémoire

[La fonction malloc]

La fonction **malloc** de la bibliothèque `<stdlib>` *permet de localiser et de réserver* de la mémoire, sa syntaxe est : **malloc(N)**

Cette fonction retourne un pointeur de type `char *` pointant vers le premier octet d'une zone mémoire libre de N octets ou le pointeur `NULL` s'il n'y a pas assez de mémoire libre à allouer.

Exemple : Si on veut réserver la mémoire pour un texte de 1000 caractères, on peut déclarer un pointeur `pt` sur **char** (`char *pt`).

L'instruction: **`T = malloc(1000);`** fournit l'adresse d'un bloc de 1000 octets libres et l'affecte à `T`. S'il n'y a pas assez de mémoire, `T` obtient la valeur zéro (`NULL`).

Remarque : Il existe d'autres fonctions d'allocation dynamique de mémoire dans la bibliothèque `<stdlib>`

[La fonction malloc et free]

- Si on veut réserver de la mémoire pour des données qui ne sont pas de type char, il faut convertir le type de la sortie de la fonction `malloc` à l'aide d'un cast.

Exemple : on peut réserver la mémoire pour 2 variables contiguës de type `int` avec l'instruction : `p = (int*)malloc(2 * sizeof(int));`

où `p` est un pointeur sur `int` (`int *p`).

- Si on n'a plus besoin d'un bloc de mémoire réservé par `malloc`, alors on peut le libérer à l'aide de la fonction `free`, dont la syntaxe est :
`free(pointeur);`
- Si on ne libère pas explicitement la mémoire à l'aide de `free`, alors elle est libérée automatiquement à la fin du programme.

[malloc et free : exemple]

Saisie et affichage d'un tableau

```
#include<stdio.h>
#include<stdlib.h>
main()
{ float *pt;
  int i,n;
  printf("Entrez la taille du tableau \n" );
  scanf(" %d" ,&n);
  pt=(float*) malloc(n*sizeof(float));
  if (pt==Null)
  {
  printf( " pas assez de mémoire \n" );
  system(" pause " );
  }
```

```
printf(" Saisie du tableau \n " );
for(i=0;i<n;i++)
{ printf ("Élément %d ? \n ",i+1);
  scanf(" %f" , pt+i);
}
printf(" Affichage du tableau \n " );
for(i=0;i<n;i++)
printf (" %f \t",*(pt+i));
free(pt);
}
```